



Electric Microbus into production

Volkswagen announces that they will begin building a new version of their ID Buzz concept bus. Hopes to start selling by 2022. <http://dgit.in/IDbuzzVW>



A new Nest thermostat

Recent leaked images suggests that the Alphabet-owned company, Nest might be announcing a new thermostat very soon. <http://dgit.in/Nestnew>

prompts, our prompting function should look like this:

```
this.prompt({
  type  : 'input',
  name  : 'username',
  message : 'What\'s your GitHub username',
  store  : true
});
```

We encourage you to add some prompts to get an idea about how different types of prompts work.

Our index.js (<http://dgit.in/IndexJS>) has a lot of code to digest. Starting with the prompt function, we are adding a prompt whenever data is required. At the end of the prompt, we handle the promise and add the answers to Yeoman's run context.

Next, at the writing function, we add the required code for copying values from source to destination.

Working with filesystems is very easy in Yeoman, because it provides a lot of utility functions in the code. The Yeoman generator context has all the functions mentioned within the code itself to make life easier.

In the writing function, you first need to prepare our package.json, then you overwrite the value in the package.json file. In Yeoman, there are mainly two contexts are defined for file operations. The first is the destination context, and the second is the template context. The destination context is just the folder where the user is running the application.

Suppose you are running your application in ~/Projects. For that project this.destinationRoot() => returns '~/Projects' and this.destinationPath('index.js') => returns '~/Projects/index.js'.

The important thing about any copy function is that when something is missing, Yeoman will create that directory or file for you. You don't have to explicitly create every file or folder, and all operations are synchronous.

In the template context similarly, this.sourceRoot() => returns

```
 './templates'
this.templatePath('package.json') => returns './templates/package.json'
```

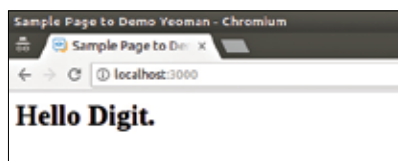
Now at line 88 you can see we have used the copyTpl method to overwrite the EJS compliant variable to assign a dynamic port number to our project here.

After copying everything the last step that remains is installing the required modules for your code to run. We are only using express, so we will install express in the end. If you need to, you can use other modules. Also, there is another feature where you can add 'save-dev': true and install some devDependencies.

In the end, to wrap up, we are sending another yosay message to the user to inform the user what else needs to be done to start the app.

If you have gotten this far, there's just one step that remains – linking the package. To do that, at the generator directory write npm link.

After linking create a test directory to test-run your generator. Now, in the test directory you will see all the files are created with the relevant values replaced. All you need to do to start the app is send out the command: npm start



The final output of the generator

IN THE END

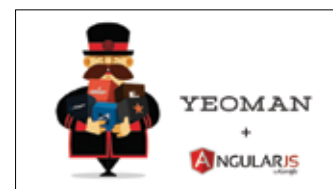
There are a lot of scaffolding tools in the market and Yeoman is only one of them. And beyond this article, there are certainly a lot of other parts to be uncovered. There are some pretty awesome generators available over there and you can publish your own awesome one too. 📌

Read More: <http://yeoman.io/>

- Inquirer: <http://dgit.in/yminquiries>
- Our Code: <http://dgit.in/ymtuts>

*POINTERS

>>Interesting developer videos



*A Yeoman tutorial

A series of videos from YouTuber Karl Hadwen that explain the concepts around Yeoman.

<http://dgit.in/YeoTutVid>



*SMS Retrieval API on Android

The process of using the SMS Retriever API on Android.

<http://dgit.in/SMSAndr>



*Life of an Indie App Developer

Marco Arment made his name developing Tumblr but is now an independent app developer.

<http://dgit.in/IndieDevlp>



*Coding Snake fassst!

Check out YouTuber Chris DeLeon code snake on plain javascript in 4min 30 secs!

<http://dgit.in/CodeSnake>